Application Overview

Main Goal

This application is a multi-vendor marketplace for bespoke tailoring. Customers can find tailors, save measurements, place and track custom orders, and review completed work. Tailors manage their shop, designs, customers, and production orders. Administrators oversee platform activity and approve shops.

How It Works

1. Customers and tailors register or sign in. The backend authenticates users and restricts dashboard and API access by role.
2. Customers browse tailor shops and designs, save garment measurements, and submit orders with customization and delivery details.
3. Tailors manage customers and move orders through production milestones; customers can follow order and delivery status.
4. The backend stores application data in a SQL database and exposes API endpoints to the React web interface. Admins manage platform operations.

What Is Implemented

- Public marketplace pages and separate customer, tailor, and admin dashboards.
- Role-based authentication and protected API access.
- English, Telugu, and Hindi interface translations; the selected language is saved between visits.
- Dark and light themes; the selected theme is saved between visits.
- Tailor discovery with specialty, city, rating, distance, and nearby-location browsing.
- Tailor shop and design management, customer records, and reusable garment measurement templates/profiles.
- Custom order creation, production milestones, delivery addresses and tracking, payment records, and reviews.
- International country/currency support, including country-aware address and phone validation and delivery fees.
- Demo accounts and sample marketplace data for trying the application locally.
- End-to-end test scripts covering security/roles, administration, delivery, navigation, internationalization, and theme behavior.

Completion Estimate

Approximately 75% of core application implementation is present. This is a rough assessment from the code and test scripts in the repository, not a measured project-tracking figure. The main user journeys and supporting API modules exist; completing the remaining work depends on the intended launch requirements.

Main Pending Work

- Connect and verify a real payment provider. The current default is `MOCK_SANDBOX`; production credentials and provider callbacks/webhooks are not demonstrated here.
- Run the full test suite against a clean environment and add repeatable automated tests for remaining API and UI flows.
- Complete production configuration: secrets, database setup/migrations, deployment, HTTPS, logging, backups, and monitoring.
- Validate real-world workflows with customers and tailors, then address usability, accessibility, and edge cases found.

Technology

- Frontend: React, TypeScript, Vite, and Tailwind CSS.
- Backend: FastAPI and SQLAlchemy, with PostgreSQL configured as the database target.

For local startup instructions, see README.md.